

CEG 4424/6424 (Security Attacks and Defenses)

Homework #2

Question 1 (10 points)

What is the difference between a block cipher and a stream cipher?

Answer: The main difference between a block cipher and a stream cipher is how they handle the data being encrypted.

A block cipher encrypts data in fixed-size chunks, such as 64 or 128 bits at a time. If the message is longer than one chunk, it gets split into multiple blocks and encrypted using a specific method. This approach mixes the data thoroughly to make it hard to read without the key.

A stream cipher, on the other hand, encrypts data one bit or byte at a time. It creates a random-looking key stream and combines it with the original data, usually using an XOR operation. Stream ciphers are often faster, but they can become less secure if the same key is reused.

Question 2 (10 points)

Suppose that you are given a plaintext message (P) and its corresponding ciphertext C using One-Time Pad with a specific key (K), i.e., $C = K \text{ XOR } P$, can you compute the One-Time Pad key (K) from P and C ? (Hint: using P and C , to compute K)

Answer: Yes, we can calculate the One-Time Pad key if you know both the plaintext and the ciphertext. A one-time pad encrypts data by combining the plaintext with the key using an XOR operation:

$$C = K \text{ XOR } P$$

XOR has a useful property: if you XOR something with the same value twice, you get the original value back. So to isolate the key, you XOR the ciphertext with the plaintext:

$$C \text{ XOR } P = (K \text{ XOR } P) \text{ XOR } P$$

The two P values cancel each other out, leaving only the key:

$$K = C \text{ XOR } P$$

In simple terms, knowing both the original message and the encrypted version lets us recover the key by XORing them together.

Question 3 (10 points)

Suppose we have a computer that can do 2^{50} AES decryptions per second. What is the expected time (in seconds) to find the correct key by brute force search if the AES key setting is 128 bits?

Answer: A 128-bit AES key means there are 2^{128} possible keys in total. Suppose a computer can try 2^{50} keys per second. A brute force attack usually finds the correct key halfway through the search, on average. So the expected time becomes half the keyspace $\div$ attempts per second $= 2^{127} \div 2^{50} = 2^{77}$ seconds.

In short, even with an extremely fast computer, the time required is astronomically large, which is why AES with a 128 bit key is considered secure against brute force attacks.

Question 4 (10 points). Show the first eight words of the key expansion for a 128-bit key 0x 0F 0E 0D 0C 0B 0A 09 08 07 06 05 04 03 02 01 00.

Answer: For a 128 bit key in Advanced Encryption Standard, the key expansion process creates 44 words, where each word is 4 bytes long. The first four words are just the original key split into pieces. Given the key: 0x0F0E0D0C0B0A09080706050403020100, we divide it into four 4-byte words:

w0 = 0F0E0D0C
w1 = 0B0A0908
w2 = 07060504
w3 = 03020100
w4 = 79726E77
w5 = 7278677F
w6 = 757E627B
w7 = 767C637B

Question 5 (20 points) Given the plaintext {000102030405060708090A0B0C0D0E0F} and the key {10101010101010101010101010101010},

Answer: We are given the plaintext {000102030405060708090A0B0C0D0E0F} and the key {10101010101010101010101010101010}. In AES, the 16 byte plaintext and key are represented as a 4x4 matrix of bytes, filled column by column.

(1) Show the original content of State, displayed as a 4*4 matrix.

Answer: The plaintext bytes are arranged into the State matrix as follows:

00	04	08	0C
01	05	09	0D
02	06	0A	0E
03	07	0B	0F

(2) Show the value of State after initial AddRoundKey.

Answer: The initial AddRoundKey is an XOR of the State matrix with the first 16 bytes of the key. The key {10101010101010101010101010101010} is also arranged into a 4x4 matrix, column by column.

Key Matrix:

Col 0	Col 1	Col 2	Col 3
10	10	10	10
10	10	10	10
10	10	10	10
10	10	10	10

XORing the two matrices byte-by-byte yields the new state:

Col 00 ¹⁰	Col 04 ¹⁰	Col 08 ¹⁰	Col 0C ¹⁰
10	14	18	1C
11	15	19	1D
12	16	1A	1E
13	17	1B	1F

(3) Show the value of State after Subbytes.

Answer: The SubBytes transformation applies a single S-box to each byte of the State. The S box is a lookup table. We will find the substitution for a few example bytes to illustrate the pattern as 0x10 from the S box becomes 0xCA, 0x14 becomes 0xFA, 0x18 becomes 0xAD, 0x1C becomes 0x9C and so on. Applying the S box to every byte results in the following State matrix:

Col 0	Col 1	Col 2	Col 3
CA	FA	AD	9C
82	59	D4	9E
C9	47	A2	72
7D	F0	AF	C0

(4) Show the value of State after Shiftrows.

Answer: The ShiftRows operation cyclically shifts the bytes in each row to the left by a specific offset.

Row 0 is shifted by 0: CA FA AD 9C → CA FA AD 9C

Row 1 is shifted by 1: 82 59 D4 9E → 59 D4 9E 82

Row 2 is shifted by 2: C9 47 A2 72 → A2 72 C9 47

Row 3 is shifted by 3: 7D F0 AF C0 → C0 7D F0 AF

This yields the final State matrix for this step:

CA	FA	AD	9C
59	D4	9E	82
A2	72	C9	47
C0	7D	F0	AF

Question 6 (10 points). With the ECB mode of DES, if there is an error in a block of the transmitted ciphertext, only the corresponding plaintext block is affected. However, in the CBC mode, this error propagates. For example, an error in the transmitted C1 obviously corrupts P1 and P2.

(a) Are any blocks beyond P2 affected if there is an error in the transmitted C1 in the CBC mode?

Answer: No, blocks beyond P2 are not affected. In CBC mode, the decryption of a block P_n depends on the ciphertext block C_n and the previous ciphertext block C_(n-1). An error in C1 will directly corrupt the decryption of P1 (because P1 = Decrypt(C1) XOR IV) and P2 (because P2 = Decrypt(C2) XOR C1). However, C2 itself is unchanged, so when it is used to decrypt P3 (P3 = Decrypt(C3) XOR C2), the result is correct. Therefore, the error propagates only to P1 and P2.

(b) Suppose that there is a bit error in the source version of P1. Through how many ciphertext blocks is this error propagated in the CBC mode?

Answer: A bit error in the source version of P1 will propagate into two ciphertext blocks: C1 and C2. In CBC encryption, C1 = Encrypt(P1 XOR IV). An error in P1 will produce a completely different C1. Then, when calculating C2 =

Encrypt($P2 \oplus C1$), the error in $C1$ (which is now part of the input for the next block) will cause $C2$ to also be different from its original value. Therefore, the error in the source plaintext affects all subsequent ciphertext blocks, as the chaining mechanism propagates the error forward indefinitely.

Question 7 (10 points). The Common Weakness Enumeration (CWE) system provides a reference method for publicly known information-security vulnerabilities and exposures. Please describe what is CWE-1240, CWE-311. (Hint: Please go to the website cve.mitre.org to find the descriptions of these CWEs. You should describe the content in your own words by rephrasing the sentences from the online resources.)

Answer: Based on the descriptions from the Common Weakness Enumeration (CWE) database:

CWE-1240: Use of a Cryptographic Primitive with a Risky Implementation is a weakness that occurs when a product needs to use a cryptographic primitive (like an encryption or hashing algorithm) but implements it in an insecure way. Instead of using a well-vetted, standardized, and compliant implementation, the developer uses one that is non-standard, unproven, or does not follow official specifications. This could involve writing a custom cryptographic function from scratch or using a library with known flaws, which can introduce vulnerabilities into the product.

CWE-311: Missing Encryption of Sensitive Data is a weakness which describes a situation where a software product fails to encrypt data that is considered sensitive or critical. The data is either stored or transmitted without any encryption, leaving it in a vulnerable state (cleartext). If an attacker gains access to the storage medium or intercepts the communication channel, they can read this sensitive information directly, bypassing any other security controls that might be in place.